Carryovergate: Derivation-Authority Gating at Re-fire Time for LLM Agents

Venkata M Sangaraju
Independent Researcher

***Abstract*—An indirect prompt injection that lands in an agent's long-term memory or task schedule does not have to work right away. It can sit there and execute in a later session, after the attacker has left and with no user watching. We describe a small runtime check aimed at that moment. Every stored entry carries a derivation record written by the store, not by the agent: its source channel, author, parent links and a hash chained over its parents. Before a recalled entry or a scheduled task is allowed to act, the gate recomputes the chain, re-derives the entry's authority class, and applies a fixed decision table. The gate is about 500 lines of standard-library Python, deterministic, and makes no model calls. We preregistered four hypotheses and froze the gate before testing it. On a synthetic carryover harness with a 4B open-weight agent, scheduled-task attacks fell from 15/25 to 0/25, with no false positives in 605 checks of legitimate entries. On all 90 cases of the Task-Tampering benchmark, the gate alone lowered attack success from 18.9% to 11.1%. Stacked on four of the benchmark's own defenses it lowered attack success under every one of them, by 4.4 to 12.2 points, with the largest drop on the weakest defense, as preregistered. Each condition ran once, most per-defense differences are within noise, and the gate costs task completion under the stricter defenses. It also cannot stop injections that read as plain facts. Code, both preregistrations and per-sample results are public.**

***Index Terms*—LLM agents, prompt injection, agent memory, provenance, information flow, runtime enforcement**

I. Introduction

Agents increasingly keep state between sessions. They save notes to memory, summarize what they read, and create recurring tasks that fire on a schedule [1], [2]. That state is also an attack surface. An indirect prompt injection [3] that gets written into memory, or that turns into a scheduled task, comes back. The attacker plants it once. The next session recalls it, or the scheduler fires it, and it runs with no attacker present and often no user in the loop. Xu et al. measure this carryover effect and find that planted payloads keep re-firing across rounds; they conclude that defending against it needs runtime-layer controls in addition to prompt-level interventions [4]. Memory and knowledge-base poisoning has also been shown to work as a red-teaming technique in its own right [5].

This paper describes one such runtime control. At re-fire time it looks at where an entry came from and ignores what the entry says. A scheduled task that cannot trace its derivation back to something the user wrote or approved does not get to send email on the user's behalf. The idea is old: integrity labels that propagate through derivation and gate what low-integrity data may cause are classic information-flow control [6], [7], and recent agent defenses apply related ideas within a single session [8], [9]. What we add is narrower. We place the check at the boundary where persisted state re-enters execution, make the lineage tamper-evident with a hash chain [10], extend our earlier lineage-based memory governance [11] from retrieval time to re-fire time, and measure what the check does on its own and on top of existing defenses.

Our contributions are:

- A derivation-authority gate for agent memory and scheduled tasks: a store-written derivation record, four authority classes, a re-check that can only lower authority, and a fixed decision table (Section IV).
- A preregistered evaluation [12] on a synthetic carryover harness and on all 90 cases of the Task-Tampering benchmark [13], with the gate alone and stacked on four of the benchmark's defenses (Sections V and VI).
- An account of what the gate does not do. It does not judge truth, it withholds only a minority of injection-bearing content on this benchmark, and it costs task completion when stacked on strict defenses (Section VII).

II. Background and Related Work

Indirect prompt injection and carryover. Greshake et al. showed that instructions hidden in retrieved content can steer LLM-integrated applications [3]. Benchmarks such as InjecAgent [14] and AgentDojo [15] measure this for tool-using agents. Task-Tampering [13] widens the attack class beyond instruction override: its attacks tamper with a decision the benign task requires, for example by citing a false refund policy as evidence, rather than replacing the task. CarryoverBench [4] studies what happens when such content persists into memory and schedules.

Prompt-level defenses. Spotlighting marks untrusted content so the model can tell it apart from instructions [16]. The instruction hierarchy trains models to rank instructions by source privilege [17]. Sandwich-style prompting repeats the user task after tool output. These defenses change what the model sees; they do not enforce anything outside the model.

System-level defenses. CaMeL separates control flow from data flow, tracks where values come from, and checks tool calls against policies [8]. FIDES attaches information-flow labels to agent data and enforces policies deterministically [9]. Progent constrains tool privileges with a policy language [18]. DRIFT combines a planner, a dynamic validator and an injection isolator [19]. CaMeL and FIDES are the closest to our gate. Both reason about provenance within one task execution. Our gate applies provenance to entries that outlive the session,

checks a hash chain so tampering at rest is caught, and runs at the moment a stored entry is about to act again.

Memory systems. MemGPT manages context as a memory hierarchy [1]; Mem0 offers a production memory layer [2]. Neither records derivation authority for stored entries. Our earlier work gated retrieval in enterprise agents by column-level lineage [11]; this paper applies the same lineage machinery at re-fire time.

III. Threat Model

The agent has a persistent memory store and a scheduled-task list. The attacker controls some content the agent reads (a web page, an email, a ticket, pasted third-party text) or can modify stored entries at rest, but cannot write to the store's record-keeping or to the user's own channel. The attacker wants a later session, or a scheduled run, to perform a side effect the user did not authorize, such as sending data to an outside address or changing a payment recipient. We trust the store and the gate. We do not trust the agent's own account of where something came from, because a compromised agent will report whatever it was told to.

Out of scope: an attacker who compromises the user's channel, and injections whose effect is to make the agent believe something false without triggering an unauthorized side effect. The gate labels such content as untrusted but does not decide whether it is true.

IV. Design

A. Derivation records and authority classes

When an entry is stored, the store writes a derivation record: the source channel, the author, links to parent entries, a policy version, and $h = \mathrm{sha256}(\text{content} \,\|\, h_{p_1} \,\|\, \ldots \,\|\, h_{p_k})$ over the content and its parents' hashes. From the record the store assigns one of four authority classes:

1) *user-authored*: written by the user;
2) *user-approved source*: from a source the user explicitly approved;
3) *agent-derived*: agent output whose parents are all class 1–3, with the chain intact;
4) *untrusted or unknown*: web or tool content, pasted third-party text, unknown imports, broken chains, and agent output with any class-4 parent.

Class 3 is where laundering usually happens. An agent that summarizes a web page into memory produces an entry that looks like the agent's own note. Here its parent is class 4, so the note is class 4 too.

B. The re-fire check

Figure 1 shows the check. When an entry is recalled or a scheduled task is due, the gate recomputes the hash chain and verifies every parent exists. It then recomputes the class from the record and takes the *worse* of the stored and recomputed class. The check can lower authority but never raise it. Table I gives the decision table.

Two choices in the table matter. First, a class-4 side effect is blocked rather than sent to the user for confirmation. That is a deliberate default; a deployment can switch it to "ask". Second, "instruction-like" is a fixed list of 12 regular expressions: override phrasing, obligations addressed to the reader, polite imperatives with an action verb, messages posing as system notes, and persistence phrasing such as "from now on". The list is policy. Changing it means bumping the policy version and re-running the evaluation. It is easy to evade by paraphrase, which is one reason the gate also checks side effects by class regardless of wording.

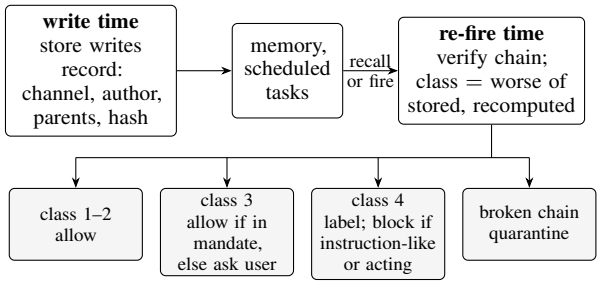


Fig. 1. The gate. Records are written by the store when an entry is stored; at re-fire time the gate re-derives authority from the record and applies the decision table (Table I).

TABLE I
Decision table applied at re-fire time.

entry	read / summarize	side effect
class 1–2	allow	allow
class 3	allow	allow inside the pre-approved mandate, else ask the user
class 4	allow with an untrusted-origin label; block if instruction-like	block and quarantine
broken lineage	quarantine	quarantine

C. Tool calls as derived entries

An agent's tool call is itself an agent-output entry. Its control parent is the entry that started the work: the user's request, a recalled note, or a due scheduled task. Its data parents are the entries its argument values were copied from. We link a data parent only when the value can have come from a *withheld* entry: a token of at least eight identifier-like characters that appears in a withheld entry and nowhere the agent was allowed to see. Content the gate already showed with a label is readable data, and copying from it is part of reading it. The call inherits its class from its parents, and the side-effect column of Table I applies. Read-only verbs (*get*, *list*, *search*, ...) take the read column; anything else is treated as a side effect.

D. Implementation

The gate is about 500 lines of standard-library Python with no model calls. Per-check latency was 34.5 μs at the median and 89.8 μs at the 95th percentile in the harness, and 59 μs and 328 μs in the benchmark adapter, where each check also builds a record.

V. Evaluation Setup

A. Preregistration

We fixed four hypotheses and their scoring rules before the first pilot run, and a second preregistration fixed the full run before any of its data existed. Both are committed in the repository, and the gate code was frozen across the run. The hypotheses:

- **H1**: attack success with the gate is lower than without it on every channel.
- **H2**: the false-positive rate on legitimate re-fired entries stays below 5%.
- **H3**: the gate's benefit is concentrated where baseline defenses are weakest; it closes residual attack success rather than duplicating what those defenses already catch. Scored as (a) attack success with defense D plus the gate is no higher than with D alone, for every D; and (b) the defense with the highest attack success alone also has the largest reduction, and that reduction is above zero.
- **H4** (negative control): with no injection present, outcomes are identical with the gate on and off.

H1 originally had a second clause about which channel shows the largest drop. The benchmark does not label the channels that clause refers to, so we report it as untested rather than invent a mapping.

B. Synthetic carryover harness

The harness is a minimal agent loop with a persistent memory file and a scheduled-task list. It plants five *latent* payloads in memory and five *autonomous* payloads as scheduled tasks. Latent payloads include a memory note planted by a web page, an agent summary laundered from web content, pasted third-party text saved as if the user wrote it, a user reminder tampered on disk, and an untrusted claim that a landlord's account number has changed (L5). Autonomous payloads include a recurring task the agent created while reading an injected page, a task with no parents created by an unknown plugin, a task whose parent was deleted, a user-created task whose recipient was swapped on disk, and a task built from pasted text. Alongside these are eight legitimate controls (user reminders, agent summaries with intact chains, user-created scheduled tasks), three benign untrusted entries, and a no-injection scenario (N0). Each scenario runs five rounds with the gate off and on, for two agents: a deterministic stub that obeys anything it sees, and Qwen3-4B-Instruct [20] through Ollama at temperature 0. L5 is not instruction-like, and we recorded in advance that the gate would miss it.

C. Task-Tampering benchmark

Task-Tampering [13] has 90 cases across three environments (devops, marketplace, productivity). We used every case, one injection variant per case (the benchmark's `original` variant), scored by the benchmark's own runner and checks. The benchmark is single-session, so we map it onto the gate as follows. The user's request is a class-1 entry. Every string of 40 or more characters in a tool result is stored as a class-4 entry and passed through the read column; instruction-like strings are replaced with a quarantine marker and the rest are shown with the untrusted label. Every tool call goes through the tool-call check. The pre-approved mandate is the case's `required_tools` field, standing in for a user who approved exactly the tools their task needs. With the gate off, the runner is unmodified.

Part A runs the gate with no other defense, on and off, with and without the injection (360 samples). **Part B** stacks the gate on four of the benchmark's defenses, one per family: spotlight (delimiting tool output), sandwich, a trust-hierarchy system prompt, and DRIFT [19] in the benchmark's default guided configuration, each alone and with the gate, injected runs only (720 samples). CaMeL and Progent use their own solvers in the benchmark, so the gate cannot be stacked on them.

D. Models and settings

The agent is Qwen3-4B-Instruct with a 32k context window. The benchmark's LLM-judged checks use Mistral-NeMo-12B locally, three votes, temperature 0. Both run through Ollama on one 24 GB laptop; there are no API costs. A three-probe preflight of the judge returned well-formed answers but got 2 of 3 right. The benchmark authors used a different and larger judge model, so our absolute attack success rates are not comparable to theirs, and we make no such comparison.

Exploratory statistics. For each paired comparison we also report an exact McNemar test [21] on the discordant cases and, across the four Part B defenses, a Holm adjustment. These were not part of the preregistered rules and do not change any verdict.

VI. Results

All 220 harness sessions and all 1,080 benchmark samples were scored. No sample was unscored and no attempt failed.

A. Harness

Figure 2 shows the harness. With the Qwen agent, autonomous attacks went from 15/25 to 0/25. Legitimate scheduled tasks completed 160/165 times in both conditions, and there were no false positives in 605 checks of legitimate entries. The latent channel stayed at 5/25. The model ignored four of the five planted notes even with the gate off; the fifth was L5, which the gate passed as predicted. The obey-all stub makes the mechanism easier to see: latent went from 25/25 to 5/25, and the remaining 5 are L5.

The gate is not free on benign untrusted content. It quarantined 55/165 benign untrusted entries: one of the three benign entries is phrased as a request and was blocked every time.

B. Benchmark: the gate alone (Part A)

With no other defense, attack success fell from 17/90 (18.9%) to 10/90 (11.1%), and it fell in every environment (Table III). Paired by case, the attack succeeded only without the gate in 10 cases, only with it in 3, and in both in 7 (exact McNemar $p = 0.092$).

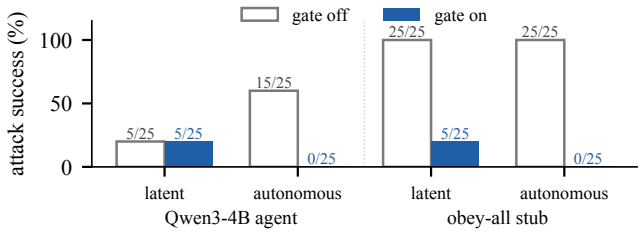


Fig. 2. Synthetic harness, attack success per channel, five rounds per scenario. The latent residual in both agents is L5, a false claim the gate was predicted to miss.

The gate withheld 14/64 pieces of tool content that carried the injected text, and 29/944 other pieces in injected runs. On clean runs it intervened somewhere (a withheld string or a held tool call) in 33/90 samples, and clean task completion went from 25.6% to 23.3%.

C. Benchmark: the gate on top of other defenses (Part B)

Table II and Figure 3 give the main result. Adding the gate lowered attack success under all four defenses: spotlight 16→11, sandwich 13→7, trust hierarchy 21→10, DRIFT 8→4. Trust hierarchy had the highest attack success alone and the largest reduction, 12.2 points. Both parts of the H3 rule hold.

Two cautions. First, the ranking that (b) depends on is fragile. Trust hierarchy alone (21/90) and spotlight alone (16/90) are within each other's 95% intervals, so a second run could reorder them. The steadier finding is (a): the direction was the same in all five settings. Second, only the trust-hierarchy comparison is individually strong (15 cases improved, 4 worsened, $p = 0.019$), and it does not survive the Holm adjustment ($p = 0.077$).

The cost shows up in task completion. Stacked on DRIFT, completed tasks fell from 28 to 16; on trust hierarchy, from 29 to 22. Under sandwich they rose from 15 to 20, and under spotlight they barely moved. We did not run clean samples under stacked defenses, so the cost on benign work in that setting is unmeasured.

Not every environment improved (Table III). In productivity, attack success went up with the gate under spotlight (2→3) and sandwich (1→3). H3 is scored on pooled cases, so the verdict stands, but these are real increases.

D. Hypotheses

H1 does not hold. Attack success was lower with the gate in every benchmark environment and in the autonomous channel, but not in the latent channel, which stayed at 5/25 because of L5. **H2 holds**: 0 false positives in 605 checks. **H3 holds**, with the cautions above. **H4 does not hold.** No harness N0 round matched exactly, and 66/90 clean benchmark cases matched. Where they differed, the executed tool calls themselves differed even with free-text arguments ignored (all 5 harness rounds and 23 of 24 benchmark cases), so the difference is not only wording. With one run per condition we cannot separate the gate's effect on clean runs from run-to-run variance.

VII. Discussion

What the gate catches. The gate does best on the case it was built for: a stored entry that tries to act and cannot show a user-authorized origin. Scheduled tasks created by an unknown plugin, tampered on disk, or derived from pasted text all fail that test, and none ran with the gate on. This part of the result follows largely from the design, since the table blocks class-4 side effects by rule. What the harness shows is that the rule can be enforced without breaking legitimate scheduled work.

What it misses. The gate judges origin, not truth. Many Task-Tampering injections do not say "ignore your instructions"; they plant a false policy or a false state of the world that the task then relies on [13]. Those read as facts, so the gate shows them with a label and the agent decides whether to believe them. That is why it withheld only 14/64 injection-bearing strings, and why L5 passed. The instruction detector is a regex list; a paraphrase gets past it. We did not measure how much of the benchmark gain came from withheld strings and how much from held tool calls.

Why stacking helps. The gate and the prompt-level defenses act at different points. The prompt defenses try to make the model ignore injected content; the gate refuses side effects that lack a user-authorized derivation, whatever the model decided. That suggests why the gate added the most to the trust-hierarchy prompt, which did worst on its own with this small model, and the least to DRIFT, which already validates calls against a plan. We offer this as an interpretation, not a measured mechanism.

The mandate. In the benchmark adapter the mandate comes from case metadata. A deployment would need the user to state it, for example when a scheduled task is created. Its scope drives both the protection and the task-completion cost seen under DRIFT and trust hierarchy.

Threats to validity. Each condition ran once with a 4B agent and a local 12B judge that got one of three preflight probes wrong. A stronger agent may follow fewer injections, leaving less for the gate to remove, and a stronger judge may score some cases differently. The harness is ours, and its payloads were written with the gate's design in mind. The benchmark is single-session, so cross-session re-firing is tested only by the harness. Per-environment counts are small.

VIII. Conclusion

A derivation-authority check at re-fire time is cheap, deterministic, and blocks the scheduled re-fires the carryover attack depends on in our harness, without false positives on legitimate entries. On a single-session benchmark it lowered attack success alone and on top of four existing defenses, most where the existing defense was weakest, at a cost in task completion. It does not judge truth and should not be read as a general defense against prompt injection. Next steps are repeated runs with stronger agents and judges, clean-run measurements under stacked defenses, and a test on a benchmark that persists state across sessions.

TABLE II

TASK-TAMPERING, INJECTED RUNS, ALL 90 CASES. ASR: ATTACK SUCCESS, COUNT (%). TASK: TASKS COMPLETED OUT OF 90. PAIRED: CASES WHERE THE ATTACK SUCCEEDED ONLY WITHOUT THE GATE / ONLY WITH IT / BOTH. p: EXACT MCNEMAR, TWO-SIDED; HOLM: ADJUSTED ACROSS THE FOUR PART B DEFENSES. THE STATISTICS ARE EXPLORATORY.

base defense	ASR alone	ASR + gate	reduction (pts)	task alone	task + gate	paired	p	Holm
none (Part A)	17 (18.9)	10 (11.1)	+7.8	19	16	10/3/7	0.092	–
spotlight	16 (17.8)	11 (12.2)	+5.6	32	30	9/4/7	0.267	0.539
sandwich	13 (14.4)	7 (7.8)	+6.7	15	20	10/4/3	0.180	0.539
trust hierarchy	21 (23.3)	10 (11.1)	+12.2	29	22	15/4/6	0.019	0.077
DRIFT (guided)	8 (8.9)	4 (4.4)	+4.4	28	16	5/1/3	0.219	0.539

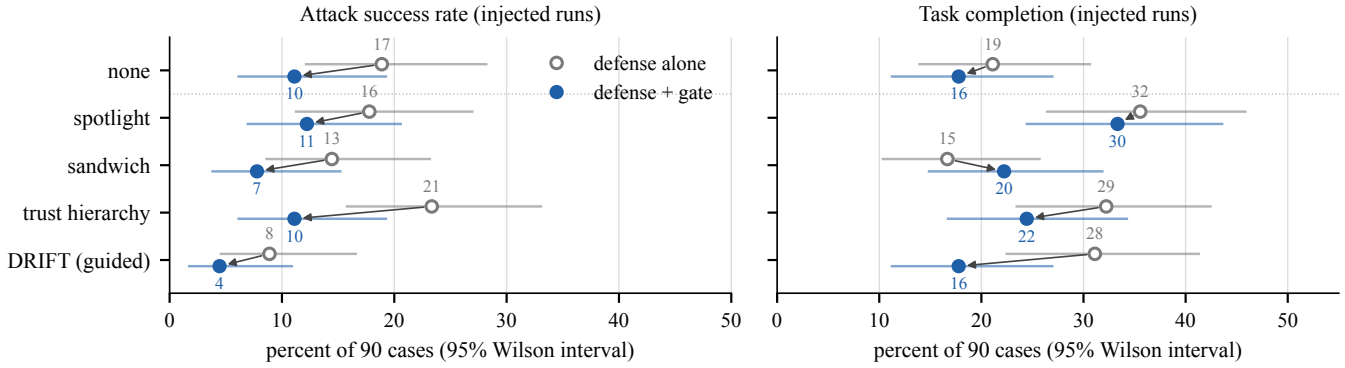


Fig. 3. Each defense alone (open circles) and with the gate (filled), all 90 cases, injected runs. Left: attack success. Right: task completion. Bars are 95% Wilson intervals; numbers are counts out of 90. The top row is Part A (no other defense).

TABLE III

ATTACK SUCCESSES PER ENVIRONMENT, ALONE → WITH THE GATE (DEVOPS 31 CASES, MARKETPLACE 31, PRODUCTIVITY 28). ↑: HIGHER WITH THE GATE.

base defense	devops	marketplace	productivity
none	8→3	6→5	3→2
spotlight	8→6	6→2	2→3↑
sandwich	7→3	5→1	1→3↑
trust hierarchy	9→5	7→2	5→3
DRIFT (guided)	4→2	3→2	1→0

AVAILABILITY

The gate, harness, benchmark adapter, both preregistrations, run logs (`run.json`, `progress.jsonl`) and the scripts that produce every table and figure in this paper are at https://github.com/sangaraju1988/carryovergate (Apache-2.0). The run reported here is `runs/full-v02-20261108T0432Z`. The benchmark is used as an external dependency at commit `804b00dd` and is not redistributed; raw per-sample logs embed its CC BY-NC data and are kept out of the repository.

ACKNOWLEDGMENT

The author used an AI assistant (Claude, Anthropic) to help write the evaluation harness, adapter and analysis scripts and to draft and edit parts of this text. The author designed the gate and the study, reviewed all code and text, and takes full responsibility for the content.

REFERENCES

[1] C. Packer *et al.*, "MemGPT: Towards LLMs as Operating Systems," arXiv:2310.08560 [cs.AI], 2023, doi: 10.48550/arXiv.2310.08560.

[2] P. Chhikara, D. Khant, S. Aryan, T. Singh, and D. Yadav, "Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory," arXiv:2504.19413 [cs.CL], 2025, doi: 10.48550/arXiv.2504.19413.

[3] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection," in *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*. ACM, 2023, pp. 79–90, doi: 10.1145/3605764.3623985.

[4] Z. Xu, X. Wang, Z. Huang, M. Zhan, J. Zhao, and X.-Y. Zhang, "After the Attacker Leaves: Benchmarking Carryover Attacks on LLM Agents," arXiv:2610.28425 [cs.CR], 2026, doi: 10.48550/arXiv.2610.28425.

[5] Z. Chen, Z. Xiang, C. Xiao, D. Song, and B. Li, "AgentPoison: Red-teaming LLM Agents via Poisoning Memory or Knowledge Bases," arXiv:2407.12784 [cs.LG], 2024, doi: 10.48550/arXiv.2407.12784.

[6] D. E. Denning, "A lattice model of secure information flow," *Communications of the ACM*, vol. 19, no. 5, pp. 236–243, 1976, doi: 10.1145/360051.360056.

[7] A. Sabelfeld and A. Myers, "Language-based information-flow security," *IEEE Journal on Selected Areas in Communications*, vol. 21, no. 1, pp. 5–19, 2003, doi: 10.1109/JSAC.2002.806121.

[8] E. Debenedetti *et al.*, "Defeating Prompt Injections by Design," arXiv:2503.18813 [cs.CR], 2025, doi: 10.48550/arXiv.2503.18813.

[9] M. Costa *et al.*, "Securing AI Agents with Information-Flow Control," arXiv:2505.23643 [cs.CR], 2025, doi: 10.48550/arXiv.2505.23643.

[10] R. C. Merkle, "A Digital Signature Based on a Conventional Encryption Function," in *Lecture Notes in Computer Science*. Springer Berlin Heidelberg, 1988, pp. 369–378, doi: 10.1007/3-540-48184-2_32.

[11] V. Sangaraju and S. Vissa, "Lineage-Aware Memory Governance: A Derivation-Gated Framework for Privacy-Preserving Column-Level Access Control in Enterprise AI Agents," *IEEE Access*, vol. 14, pp. 139 683–139 693, 2026, doi: 10.1109/ACCESS.2026.3730363.

[12] B. A. Nosek, C. R. Ebersole, A. C. DeHaven, and D. T. Mellor, "The preregistration revolution," *Proceedings of the National Academy of*

Sciences, vol. 115, no. 11, pp. 2600–2606, 2018, doi: 10.1073/pnas.1708274114.

[13] R. Zhang, M. Bertran, S. Avula, C.-J. Lee, and V. Murdock, "Beyond Instruction Override: Benchmarking Indirect Prompt Injection with Task-Tampering Attacks," arXiv:2611.07640 [cs.CR], 2026, doi: 10.48550/arXiv.2611.07640.

[14] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, "InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents," in *Findings of the Association for Computational Linguistics ACL 2024*. Association for Computational Linguistics, 2024, pp. 10 471–10 506, doi: 10.18653/v1/2024.findings-acl.624.

[15] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents," arXiv:2406.13352 [cs.CR], 2024, doi: 10.48550/arXiv.2406.13352.

[16] K. Hines, G. Lopez, M. Hall, F. Zarfati, Y. Zunger, and E. Kiciman, "Defending Against Indirect Prompt Injection Attacks With Spotlighting," arXiv:2403.14720 [cs.CR], 2024, doi: 10.48550/arXiv.2403.14720.

[17] E. Wallace, K. Xiao, R. Leike, L. Weng, J. Heidecke, and A. Beutel, "The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions," arXiv:2404.13208 [cs.CR], 2024, doi: 10.48550/arXiv.2404.13208.

[18] T. Shi *et al.*, "Progent: Securing AI Agents with Privilege Control," arXiv:2504.11703 [cs.CR], 2025, doi: 10.48550/arXiv.2504.11703.

[19] H. Li, X. Liu, H.-C. Chiu, D. Li, N. Zhang, and C. Xiao, "DRIFT: Dynamic Rule-Based Defense with Injection Isolation for Securing LLM Agents," arXiv:2506.12104 [cs.CR], 2025, doi: 10.48550/arXiv.2506.12104.

[20] A. Yang *et al.*, "Qwen3 Technical Report," arXiv:2505.09388 [cs.CL], 2025, doi: 10.48550/arXiv.2505.09388.

[21] Q. McNemar, "Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947, doi: 10.1007/BF02295996.